Name: R. Kishan Reg No: 192525378

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I R. Kishan (192525378) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

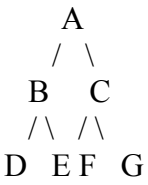
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

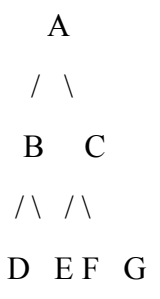
Iteration	Queue	Visited Node	
1	B,C	**A**	Dequeue A, visit A, enqueue B, C
2	C,D,E	**B**	Dequeue B, visit B, enqueue D, E
3	D,E,F,G	**C**	Dequeue C, visit C, enqueue F, G
4	E,F,G	**D**	Dequeue D, visit D (leaf)
5	F,G	**E**	Dequeue E, visit E (leaf)
6	G	**F**	Dequeue F, visit F (leaf)
7	empty	**G**	Dequeue G, visit G (leaf), queue empty

Questions:

1. Write the BFS traversal order. **BFS order: A → B → C → D → E → F → G**
2. Show the queue contents after each iteration.

Queue after each iteration: [B,C] → [C,D,E] → [D,E,F,G] → [E,F,G] → [F,G] → [G] → [ ]

2. Depth-First Search (DFS) Using the same graph,



(stack top = right end; children pushed right to left)

Pop A, visit A, push C then B
Pop B, visit B, push E then D
Pop D, visit D (leaf)
Pop E, visit E (leaf)
Pop C, visit C, push G then F
Pop F, visit F (leaf)
Pop G, visit G (leaf), stack empty

Iteration	Stack	Visited Node
1	C,B	**A**
2	C,E,D	**B**
3	C,E	**D**
4	C	**E**
5	G,F	**C**
6	G	**F**
7	empty	**G**

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table. **DFS order: A → B → D → E → C → F → G**

3. **Initial State :**

1 3 6
5 0 2
4 7 8

Goal State :

1	**2**	**3**
4	**5**	**6**
7	**8**	**0**

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	S0 136/502/478	S0	S1 S2 S3 S4
2	S1 106/532/478	S1 S2 S3 S4	S2 S3 S4 S5 S6
3	S2 136/572/408	S2 S3 S4 S5 S6	S3 S4 S5 S6 S7 S8
4	S3 136/052/478	S3 S4 S5 S6 S7 S8	S4 S5 S6 S7 S8 S9 S10
5	S4 136/520/478	S4 S5 S6 S7 S8 S9 S10	S5 S6 S7 S8 S9 S10 S11 S12

Expanded in iterations 1-5: S0, S1, S2, S3, S4. (Each state is shown as 3 rows separated by '/', blank = 0.)

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

State IDs used in the table (blank tile = 0). Blank moves tried in order Up, Down, Left, Right; repeated states are not queued again.

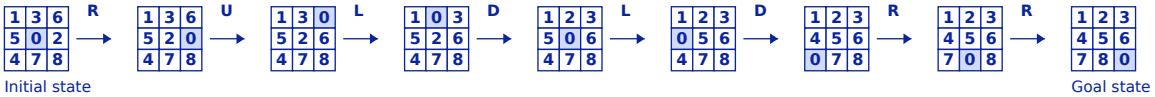
S0 = 136/502/478	S1 = 106/532/478	S2 = 136/572/408	S3 = 136/052/478	S4 = 136/520/478
S5 = 016/532/478	S6 = 160/532/478	S7 = 136/572/048	S8 = 136/572/480	S9 = 036/152/478
S10 = 136/452/078	S11 = 130/526/478	S12 = 136/528/470		

ANSWERS

1. Iterations 1-5 expand S0, S1, S2, S3, S4 in that order (FIFO: the oldest state in the queue is always expanded first). The goal state is generated as S310 and is expanded in iteration 311.

2. 500 states are generated in total (1 start + 499 new states) by the time the goal is taken from the queue; 311 are expanded. By depth: 1 + 4 + 8 + 8 + 16 + 32 + 60 + 72 + 136 + 163 = 500 (depth 0 to 9, repeated states discarded).

3. Solution path (blank moves): Right, Up, Left, Down, Left, Down, Right, Right = 8 moves.



4. BFS uses a FIFO queue, so every state at depth d is expanded before any state at depth d+1. In an unweighted 8-puzzle every move costs 1, so path cost = depth. The first time the goal is reached is therefore at the smallest possible depth (8 moves here); BFS is complete and optimal, and no shorter solution can exist.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

Answer Sheet (10 marks per question)

Q1. Breadth-First Search (BFS)

Aim: traverse the tree from A level by level. **Data structure:** FIFO queue. **Rule:** dequeue the front node, visit it, enqueue its children left to right.

Iter	Queue (after iteration)	Visited node	Action
1	B, C	A	Dequeue A, enqueue B, C
2	C, D, E	B	Dequeue B, enqueue D, E
3	D, E, F, G	C	Dequeue C, enqueue F, G
4	E, F, G	D	Dequeue D (leaf)
5	F, G	E	Dequeue E (leaf)
6	G	F	Dequeue F (leaf)
7	empty	G	Dequeue G (leaf), queue empty

Answer 1 (BFS order): A → B → C → D → E → F → G
Answer 2 (queue contents): [B,C] → [C,D,E] → [D,E,F,G] → [E,F,G] → [F,G] → [G] → [ ]
Result: all 7 nodes are visited level by level (A | B, C | D, E, F, G). The queue is empty, so the search ends.

Q2. Depth-First Search (DFS)

Aim: go as deep as possible along a branch before backtracking. **Data structure:** LIFO stack (top = right end). **Rule:** pop the top node, visit it, push its children right to left so the left child is explored first.

Iter	Stack (after iteration)	Visited node	Action
1	C, B	A	Pop A, push C then B
2	C, E, D	B	Pop B, push E then D
3	C, E	D	Pop D (leaf)
4	C	E	Pop E (leaf)
5	G, F	C	Pop C, push G then F
6	G	F	Pop F (leaf)
7	empty	G	Pop G (leaf), stack empty

DFS order: A → B → D → E → C → F → G
Difference from BFS: DFS finishes the whole left subtree (B, D, E) before moving to C, whereas BFS visits B and C first. DFS needs less memory but does not guarantee the shortest path.

Q3. 8-Puzzle using BFS

Problem: Initial state 136 / 502 / 478 (0 = blank). Goal state 123 / 456 / 780. **Operators:** move the blank Up, Down, Left, Right (tried in this order). **Step cost:** 1. **Search:** FIFO queue; repeated states are not queued again.

State labels: S0 = 136/502/478, S1 = 106/532/478, S2 = 136/572/408, S3 = 136/052/478, S4 = 136/520/478, S5 = 016/532/478, S6 = 160/532/478, S7 = 136/572/048, S8 = 136/572/480, S9 = 036/152/478, S10 = 136/452/078, S11 = 130/526/478, S12 = 136/528/470 (rows separated by /).

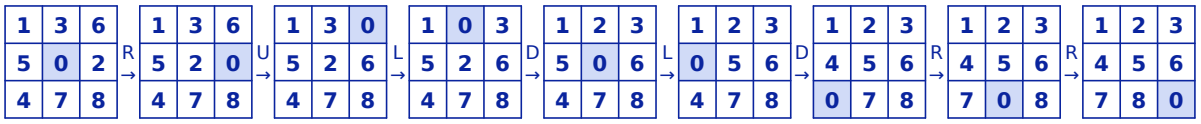
Iter	Current state	Queue before expansion	Queue after expansion
1	S0	S0	S1 S2 S3 S4
2	S1	S1 S2 S3 S4	S2 S3 S4 S5 S6
3	S2	S2 S3 S4 S5 S6	S3 S4 S5 S6 S7 S8
4	S3	S3 S4 S5 S6 S7 S8	S4 S5 S6 S7 S8 S9 S10
5	S4	S4 S5 S6 S7 S8 S9 S10	S5 S6 S7 S8 S9 S10 S11 S12

Children generated: S0 → S1 (Up), S2 (Down), S3 (Left), S4 (Right). S1 → S5, S6. S2 → S7, S8. S3 → S9, S10. S4 → S11, S12. The move back to the parent is a repeated state and is skipped. BFS continues in the same way until the goal is dequeued.

Answer 1 – Nodes expanded: S0, S1, S2, S3, S4 in iterations 1 to 5 (oldest state first). The goal state (S310) is expanded in iteration 311.

Answer 2 – States generated: 500 in total (1 start + 499 new), with 311 expanded, when the goal is taken from the queue. Count by depth 0 to 9: 1 + 4 + 8 + 8 + 16 + 32 + 60 + 72 + 136 + 163 = 500.

Answer 3 – Solution path (blank moves): Right, Up, Left, Down, Left, Down, Right, Right = 8 moves, cost 8.



Answer 4 – Why BFS gives the shortest solution:

1. BFS expands states level by level with a FIFO queue, so all states at depth d are expanded before any state at depth d+1.
2. In the unweighted 8-puzzle every move costs 1, so the path cost of a state equals its depth.
3. Hence the first time the goal is reached it is at the smallest possible depth (8 moves here), and no shorter solution exists.
4. BFS is complete (the branching factor is finite) and optimal for equal step costs. Its drawback is memory, because it stores every state of a level.